12. Write a Java program to demonstrate:

a) Threading

b) Multi-threading

12(a). Threading

Aim

To write a Java program to demonstrate Threading.

Theory

A thread is a lightweight sub-process. In Java, threading allows a program to execute multiple tasks concurrently.

A thread can be created by extending the Thread class and overriding the run() method.

When the start() method is called, the thread begins execution.

Algorithm

1. Create a class that extends the Thread class.
2. Override the run() method.

3. Create an object of the thread class.

4. Call the start() method to begin execution.

Program Code

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }

    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
    }
}
```

Result

The program successfully creates and executes a single thread, demonstrating Threading in Java.

12(b). Multi-threading

Aim

To write a Java program to demonstrate Multi-threading.

Theory

Multithreading allows multiple threads to run concurrently within a single program.
Each thread performs a separate task, improving performance and efficient CPU utilization.
In Java, multithreading can be implemented by creating multiple thread objects.

Algorithm

1. Create a class that extends the Thread class.
2. Override the run() method.
3. Create multiple objects of the thread class.
4. Call the start() method for each thread.

Program Code

```
class ThreadOne extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Thread One: " + i);
        }
    }
}

class ThreadTwo extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Thread Two: " + i);
        }
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        ThreadOne t1 = new ThreadOne();
        ThreadTwo t2 = new ThreadTwo();

        t1.start();
        t2.start();
```

```
    }
}
```

Result

Multiple threads execute simultaneously, producing interleaved output, thereby demonstrating Multi-threading in Java.

13. Write a Java program to demonstrate Exception Handling using Throw and Throws. The try and catch block utilization needs to be implemented in each case.

Aim

To write a Java program to demonstrate Exception Handling using throw and throws keywords along with try and catch blocks.

Theory

Exception Handling in Java is used to handle runtime errors and maintain normal program flow.

- throw keyword is used to explicitly throw an exception.
- throws keyword is used to declare exceptions that may occur in a method.
- try block contains code that may generate an exception.
- catch block handles the exception.

Algorithm

1. Create a method that checks a condition.
2. Use throw to generate an exception manually.
3. Use throws to declare the exception.
4. Enclose method call inside a try block.
5. Handle the exception using a catch block.
6. Display appropriate messages.

Program Code

```
class ThrowThrowsDemo {

    static void checkAge(int age) throws ArithmeticException {
        if (age < 18) {
            throw new ArithmeticException("Not eligible to vote");
        } else {
            System.out.println("Eligible to vote");
        }
    }

    public static void main(String[] args) {
        try {
            checkAge(16);
        }
        catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
        finally {
            System.out.println("Exception handling completed");
        }
    }
}
```

Result

The program successfully demonstrates Exception Handling using throw and throws.
The exception is generated, caught, and handled using try and catch blocks, ensuring smooth execution of the program.

14. Write a Java program to implement the InputStream and OutputStream classes.

Aim

To write a Java program to implement InputStream and OutputStream classes.

Theory

In Java, InputStream and OutputStream are abstract classes used for byte-oriented input and output operations.

- InputStream is used to read data from a source such as a file.
- OutputStream is used to write data to a destination.
- FileInputStream and FileOutputStream are commonly used subclasses.

These classes are mainly used for handling binary data and file operations.

Algorithm

1. Create a file output stream to write data into a file.
2. Write a string into the file using FileOutputStream.
3. Close the output stream.

4. Create a file input stream to read data from the file.
5. Read the contents of the file.
6. Display the data on the screen.

Program Code

```
import java.io.*;

class InputOutputStreamDemo {
   public static void main(String[] args) {
      try {
         // Writing data to file
         FileOutputStream fout = new FileOutputStream("sample.txt");
         String data = "Java InputStream and OutputStream Example";
         byte[] b = data.getBytes();
         fout.write(b);
         fout.close();

         // Reading data from file
         FileInputStream fin = new FileInputStream("sample.txt");
         int i;
         System.out.print("File contents: ");
         while ((i = fin.read()) != -1) {
            System.out.print((char) i);
         }
         fin.close();
      }
      catch (IOException e) {
```

```
            System.out.println("Exception occurred: " + e);
        }
    }
}
```

Result

The program successfully demonstrates the use of InputStream and OutputStream classes by writing data to a file and reading it back.